

TCGA BRCA ssGSEA Pipeline

Step-by-Step Scientific Explanation

Reactome Pathway Scoring with 8-Phase Clinical Stage Encoding
Based on Barbie et al. 2009

Dataset	TCGA BRCA (HiSeqV2 RNA-seq)
Method	Single-sample GSEA (ssGSEA)
Pathways	Reactome (GMT format)
Labels	8-phase binary stage (TMT $\times$ RT $\times$ OS)

Contents

Overview	2
1 Step 1 — Download TCGA BRCA Gene Expression	3
2 Step 2 — Fetch Reactome Pathways (GMT Format)	3
3 Step 3 — Pre-compute Gene Indices and Pathway Masks	4
4 Step 4 — Rank All Samples	5
5 Step 5 — ssGSEA Scoring Function	6
6 Step 6 — Score All Pathways in Parallel Batches	8
7 Step 7 — Build and Normalise the Pathway Score Matrix	9
8 Step 8 — Download TCGA BRCA Clinical Matrix	10
9 Step 9 — Build the 8-Phase Stage Column	10
10 Step 10 — Merge Stage Labels with Pathway Scores	12
11 Step 11 — Robust Download with 3-Method Fallback Chain	13
Summary Table	14

Overview

This pipeline takes raw TCGA breast cancer (BRCA) gene expression data and produces a final matrix where every row is a patient sample and every column is either a clinical label (**stage**) or a Reactome pathway score. The pipeline has **11 steps**:

1. Download TCGA BRCA RNA-seq gene expression
2. Fetch Reactome pathway definitions (GMT format)
3. Pre-compute gene indices and boolean pathway masks
4. Rank all samples using `scipy.rankdata`
5. Vectorised ssGSEA scoring function (Barbie et al. 2009)
6. Score all pathways in parallel batches
7. Build normalised $[0, 1]$ pathway score matrix
8. Download TCGA BRCA clinical phenotype matrix
9. Build the 8-phase stage label column
10. Merge stage labels with pathway scores (common samples)
11. Robust download with 3-method fallback chain

Big Picture — Plain English

This pipeline takes raw gene expression data from ~1,100 breast cancer patients and converts it into a machine learning-ready dataset.

Raw data has ~20,000 genes per patient — too many to work with directly. The pipeline groups those genes into ~1,400 biological pathways and computes an activity score for each pathway in each patient. It then assigns every patient a clinical label (Stage 0–7) based on their treatment history and survival time.

The final output is one table: each row is a patient, the first column is their stage label, and the remaining columns are pathway activity scores. This is ready to feed directly into a machine learning model.

1. Step 1 — Download TCGA BRCA Gene Expression

Step 1: Load the expression matrix

What it does: Downloads the TCGA BRCA HiSeqV2 $\log_2(\text{RPKM}+1)$ gene expression matrix directly from the Xena Hub. Each row is a gene; each column is a patient sample (tumour biopsy). Missing values are imputed with the per-gene median.

```
1 url = "https://tcga-xena-hub.s3.us-east-1.amazonaws.com/
   download/
   TCGA.BRCA.sampleMap%2FHiSeqV2.gz"
2
3 df = pd.read_csv(url, sep="\t", index_col=0, compression="
   gzip")
4 df = df.apply(pd.to_numeric, errors="coerce")
5 df = df.T.fillna(df.median(axis=1)).T
```

Why this choice?

- **Why HiSeqV2?** It is the primary TCGA BRCA Illumina HiSeq RNA-seq dataset — the same one used in the TCGA Nature 2012 paper, making results directly comparable to published literature.
- **Why median imputation?** Median is robust to outliers. For RNA-seq, a small number of extreme values (outlier tumours) can skew the mean. Replacing missing entries with the per-gene median preserves the expression distribution without introducing bias.
- **Why `pd.to_numeric(errors='coerce')`?** Text artefacts (e.g. “NA”, “-”) in the raw file would cause downstream arithmetic errors. Coercing them to NaN then imputing ensures a clean numeric matrix.

Big Picture — Plain English

This is the starting point of the entire pipeline. It loads the gene expression table — a matrix of $\sim 20,000$ genes across $\sim 1,100$ BRCA tumour samples. Every number in this table represents how active a gene is in one patient’s tumour. Without this data, no further analysis is possible.

2. Step 2 — Fetch Reactome Pathways (GMT Format)

Step 2: Load pathway gene sets

What it does: Downloads the Reactome GMT (Gene Matrix Transposed) file. Each line in a GMT file is one pathway:

Pathway Name	URL/Description	g_1	g_2	$\dots$	g_k
col 1	col 2	gene list			

After loading, pathways are filtered to those with **10–1000 genes**.

```

1 gene_sets = {
2     name: genes for name, genes in gene_sets_raw.items()
3     if 10 <= len(genes) <= 1000
4 }

```

Why this choice?

Why minimum 10 genes?

A pathway with fewer than 10 genes is *statistically unstable*. The ssGSEA enrichment score depends on comparing two empirical cumulative distribution functions (ECDFs). With $k < 10$ genes, the “in-pathway” ECDF has too few steps to distinguish a real signal from random variation. This is the standard threshold used in the original GSEA paper (Subramanian et al. 2005) and retained in all ssGSEA implementations.

Why maximum 1000 genes?

Very large gene sets (e.g. generic housekeeping pathways with 3000+ genes) tend to be non-specific — they are enriched in almost every sample and therefore carry no discriminative information. Furthermore, they disproportionately inflate computation time. The 1000-gene ceiling removes these uninformative sets while keeping all biologically specific pathways.

Filter	Value	Reason
Minimum genes	10	Statistical stability of ECDF
Maximum genes	1000	Remove non-specific housekeeping sets

Big Picture — Plain English

Genes do not act in isolation — they work together in biological pathways (e.g. DNA repair, immune signalling, cell cycle). Reactome is a curated scientific database that lists which genes belong to which pathway. This step downloads those pathway definitions so the pipeline knows how to group the 20,000 genes into ~1,400 meaningful biological processes. The size filter removes pathways that are too small to measure reliably or too large to mean anything specific.

3. Step 3 — Pre-compute Gene Indices and Pathway Masks

Step 3: Build boolean masks for each pathway

What it does: Creates a boolean array of length n_{genes} for each pathway. Position i is **True** if gene i belongs to that pathway, **False** otherwise.

```

1 mask = np.zeros(n_genes, dtype=bool)
2 for g in overlap:

```

```

3     mask[gene_index[g]] = True
4     pathway_masks[name] = mask

```

Why this choice?

- **Why pre-compute masks instead of filtering on the fly?** If masks are built inside the scoring loop, the same string-lookup and array-creation operations repeat for every pathway $\times$ every sample. Pre-computing once and reusing saves $\mathcal{O}(P \cdot N)$ redundant operations (where P = pathways, N = samples).
- **Why boolean (not integer)?** NumPy boolean arrays use 1 byte per element instead of 8 bytes for `float64`, reducing memory by $8\times$. Boolean indexing is also vectorised and faster than integer fancy indexing for this use case.

Big Picture — Plain English

For each pathway, we mark which genes belong to it and which do not. This marking is done once here and reused in every calculation later, so we do not repeat the same lookup thousands of times.

4. Step 4 — Rank All Samples

Step 4: Convert expression values to ranks

What it does: For every sample s , replaces each gene's expression value x_{gs} with its rank among all genes in that sample. `method='average'` assigns tied values the mean of their ranks.

```

1 def rank_one_sample(col_idx):
2     return rankdata(expr_arr[:, col_idx], method="average")

```

After ranking, the columns are sorted in *descending* rank order once (globally), producing `global_order` and `sorted_ranks` — reused by every pathway.

Why this choice?

Why rank instead of raw expression values?

ssGSEA is a *rank-based* method by design (Barbie et al. 2009). Ranking achieves two goals:

1. **Normalisation:** Removes differences in library size and dynamic range between samples. After ranking, all samples are on the same $[1, n_{\text{genes}}]$ scale.
2. **Robustness:** Extreme outlier genes (very high or very low expression) do not disproportionately drive the score because only their rank — not their absolute value — matters.

Why `method='average'` for ties? The “average” method is the exact spec-

ification in the original Barbie et al. 2009 paper. Using “min” or “max” would introduce systematic bias for genes with identical expression (common in RNA-seq for zero-count genes).

Why sort descending once globally? The enrichment score requires genes ordered from highest to lowest rank. Sorting once and storing the result avoids P redundant $\mathcal{O}(n \log n)$ sorts (one per pathway).

Big Picture — Plain English

Different patients’ samples are measured under different lab conditions, so raw expression numbers cannot be directly compared. Instead of using the raw number, we use the gene’s position (rank) within that patient — rank 1 means most active gene, rank 20,000 means least active. This puts all patients on the same scale so they can be fairly compared.

5. Step 5 — ssGSEA Scoring Function

Step 5: Vectorised ssGSEA (Barbie et al. 2009)

What it does: For each pathway, computes an enrichment score per sample by comparing the weighted ECDF of *in-pathway* genes against the unweighted ECDF of *out-of-pathway* genes.

```

1 def ssgsea_vectorized(mask, alpha=0.25):
2     ranked_mask = mask[global_order]
3     in_weights = np.where(ranked_mask, sorted_ranks ** alpha
4         , 0.0)
5     in_sum = in_weights.sum(axis=0, keepdims=True) + 1e
6         -12
7     ecdf_in = np.cumsum(in_weights, axis=0) / in_sum
8     out_count = (~ranked_mask).sum(axis=0, keepdims=True) +
9         1e-12
10    ecdf_out = np.cumsum(~ranked_mask, axis=0) / out_count
11    return (ecdf_in - ecdf_out).sum(axis=0)

```

Formula

Let:

- $r_i^{(s)}$ = rank of gene i in sample s (descending order)
- $\mathcal{G}$ = set of genes in the pathway
- $\alpha = 0.25$ = weighting exponent

Weighted in-pathway ECDF at position j :

$$P_{\mathcal{G}}(j) = \frac{\sum_{i=1}^j \mathbf{1}[i \in \mathcal{G}] r_i^{\alpha}}{\sum_{i=1}^n \mathbf{1}[i \in \mathcal{G}] r_i^{\alpha}}$$

Unweighted out-of-pathway ECDF at position j :

$$P_{\bar{\mathcal{G}}}(j) = \frac{\sum_{i=1}^j \mathbf{1}[i \notin \mathcal{G}]}{|\bar{\mathcal{G}}|}$$

ssGSEA enrichment score for sample s :

$$\text{ES}_{\mathcal{G}}^{(s)} = \sum_{j=1}^n \left[P_{\mathcal{G}}(j) - P_{\bar{\mathcal{G}}}(j) \right]$$

Why this choice?

Why $\alpha = 0.25$?

The exponent α controls how much higher-ranked genes are up-weighted relative to lower-ranked ones.

α	Effect	Use case
0	All in-pathway genes weighted equally	Classic GSEA
0.25	Mild preference for top-ranked genes	ssGSEA default
1	Strong preference for top-ranked genes	Aggressive weighting

$\alpha = 0.25$ is the exact value specified by Barbie et al. (2009) and is the community standard for ssGSEA. It provides enough sensitivity to top-ranked genes without completely ignoring mid-ranked pathway members.

Why add 1e-12 (epsilon)? Prevents division by zero if a pathway has zero overlap with the expression matrix. This is a numerical safeguard, not a mathematical modification.

Big Picture — Plain English

For each patient and each pathway, this step asks: are the genes of this pathway ranked near the top (highly active) or scattered randomly? If they are near the top, the pathway gets a high score. If scattered, the score is low. This produces one activity score per patient per pathway — converting 20,000 genes into ~1,400 pathway scores per patient.

6. Step 6 — Score All Pathways in Parallel Batches

Step 6: Parallel batch scoring

What it does: Groups pathways into batches of 100 and distributes them across all available CPU cores using `joblib.Parallel`.

```
1 BATCH_SIZE = 100
2 batches = [
3     list(pathway_masks.items())[i : i + BATCH_SIZE]
4     for i in range(0, len(pathway_masks), BATCH_SIZE)
5 ]
6 batch_results = Parallel(n_jobs=-1, verbose=2)(
7     delayed(score_batch)(batch) for batch in batches
8 )
```

Why this choice?

Why `n_jobs=-1`? Tells `joblib` to use *all* available CPU cores automatically. This is safe because each pathway is scored independently (no shared mutable state).

Why batch size 100?

- Too small (e.g. 1): Parallelisation overhead dominates — spawning a process per pathway is slower than single-threaded.
- Too large (e.g. all pathways in one batch): One core does all the work; no parallelism benefit.
- 100 is a practical sweet spot — large enough to amortise overhead, small enough for fine-grained load balancing across cores.

Does batching change the mathematics? No. Each pathway's score depends only on its own mask and the pre-computed global rank matrix. Batching is purely an engineering decision — the output is bit-for-bit identical to a single sequential loop.

Big Picture — Plain English

There are ~1,400 pathways to score across ~1,100 patients. Running them one at a time would take hours. This step runs multiple pathways simultaneously by distributing the work across all available CPU cores. The results are identical to running them sequentially — only the time required is reduced.

7. Step 7 — Build and Normalise the Pathway Score Matrix

Step 7: Assemble and normalise to [0, 1]

What it does: Stacks all pathway scores into a matrix (pathways × samples), then applies per-pathway min–max normalisation.

```
1 pathway_df = pathway_df.subtract(pathway_df.min(axis=1), axis=0)
2 pathway_df = pathway_df.divide(
3     pathway_df.max(axis=1).replace(0, 1), axis=0
4 )
```

Formula

For each pathway p and sample s :

$$\hat{E}_{p,s} = \frac{E_{p,s} - \min_s(E_{p,\cdot})}{\max_s(E_{p,\cdot}) - \min_s(E_{p,\cdot})} \in [0, 1]$$

Why this choice?

Why normalise to [0, 1]?

Raw ssGSEA scores have different scales for different pathways: a large pathway (500 genes) produces larger absolute scores than a small pathway (15 genes) simply because there are more cumulative differences to sum. Min–max normalisation puts all pathways on a common [0, 1] scale, making them directly comparable and preventing large pathways from dominating downstream analyses (e.g. machine learning models).

Why .replace(0, 1) before dividing? If a pathway has identical scores across all samples ($\max - \min = 0$), dividing by zero would produce NaN. Replacing 0 with 1 yields a constant row of 0s — a safe and interpretable result (no variation = no information).

Big Picture — Plain English

The raw ssGSEA scores from Step 6 are not directly comparable across pathways — a larger pathway naturally produces a larger number. This step rescales every pathway's scores to the range [0, 1], where 0 is the least active patient and 1 is the most active patient for that pathway. After this, all pathways are on the same scale and can be fairly compared in any downstream model.

8. Step 8 — Download TCGA BRCA Clinical Matrix

Step 8: Load clinical phenotype data

What it does: Downloads the TCGA BRCA clinical matrix from the Xena Hub. This file contains 194 clinical variables for each patient sample including treatment history, survival times, and pathological features.

```
1 clin_url = (
2     "https://tcga-xena-hub.s3.us-east-1.amazonaws.com/
3     download/"
4     "TCGA.BRCA.sampleMap%2FBRCA_clinicalMatrix"
5 )
6 clin_df = pd.read_csv(clin_url, sep="\t", index_col=0,
7                       low_memory=False)
```

Why this choice?

Why `low_memory=False`? The clinical matrix has 194 mixed-type columns. Without this flag, pandas infers column dtypes chunk by chunk, which can assign inconsistent types (e.g. a column read as `int` in one chunk and `str` in another). `low_memory=False` forces a full pass, guaranteeing consistent dtypes at the cost of slightly more memory.

Big Picture — Plain English

Steps 1–7 produced pathway scores from gene expression data. This step loads the clinical side of the data: what treatments each patient received, how long they survived, and other medical information. This file is needed to build the outcome labels (Stage 0–7) that will be attached to the pathway scores in Steps 9 and 10.

9. Step 9 — Build the 8-Phase Stage Column

Step 9: Encode three clinical variables into a stage (0–7)

What it does: Three clinical columns are each binarised, then combined into a single decimal integer (`stage`) representing all $2^3 = 8$ combinations.

```
1 stage_df["tmt_bin"] = ...map({"YES": 1, "NO": 0})
2 stage_df["rt_bin"]  = ...map({"YES": 1, "NO": 0})
3 stage_df["os_bin"]  = (OS_Time_nature2012 >= 180).astype(int)
4 stage_df["stage"]   = tmt_bin*4 + rt_bin*2 + os_bin*1
```

Formula

Bit assignment (MSB to LSB):

$$\underbrace{b_{\text{TMT}}}_{\text{bit 2}} \quad \underbrace{b_{\text{RT}}}_{\text{bit 1}} \quad \underbrace{b_{\text{OS}}}_{\text{bit 0}}$$

$$\text{stage} = 4 \cdot b_{\text{TMT}} + 2 \cdot b_{\text{RT}} + 1 \cdot b_{\text{OS}}$$

Stage	Binary	TMT	RT	OS ($\geq 180\text{d}$)
0	000	No	No	No
1	001	No	No	Yes
2	010	No	Yes	No
3	011	No	Yes	Yes
4	100	Yes	No	No
5	101	Yes	No	Yes
6	110	Yes	Yes	No
7	111	Yes	Yes	Yes

Why this choice?

Why these three variables?

- **Targeted Molecular Therapy (TMT):** Captures whether the patient received precision medicine (e.g. trastuzumab for HER2+). This is a key determinant of molecular pathway activity.
- **Radiation Therapy (RT):** Captures loco-regional treatment. Radiation activates DNA damage response pathways, which are directly measurable in the ssGSEA scores.
- **OS Time ≥ 180 days:** Separates early-mortality (< 6 months) from longer-term survivors. 180 days is a clinically established early-event threshold used in BRCA prognosis studies.

Why drop samples missing any of the three? A sample missing any one column cannot be assigned an unambiguous stage label. Using it would silently introduce label noise into downstream supervised models.

Why binary encoding into a decimal stage? Treating the three bits as a single integer provides a compact, unambiguous label for each of the 8 clinical phenotype groups, suitable as a multi-class target variable in machine learning.

Big Picture — Plain English

Each patient is assigned a single label (0–7) based on three yes/no clinical facts: did they receive targeted therapy, did they receive radiation, and did they survive at least 180 days. The three yes/no answers form a 3-bit binary number that is converted to a decimal stage value. This gives 8 distinct clinical groups. Any

patient with missing data in any of the three columns is excluded so that every label is unambiguous.

10. Step 10 — Merge Stage Labels with Pathway Scores

Step 10: Inner join on common sample IDs

What it does: Joins the stage labels (rows = samples with valid stage) and the pathway score matrix (rows = all expression samples) using an *inner join* — keeping only samples present in both.

```
1 combined = stage_df.join(pathway_T, how="inner")
2 # Final: (common samples) x (1 stage column + P pathway
   columns)
```

Why this choice?

Why inner join (not left or outer)?

Join type	Result	Problem
Left join	All stage samples	Missing pathway scores → NaN features
Right join	All pathway samples	Missing stage labels → unlabelled rows
Outer join	All samples	Both problems above
Inner join	Intersection only	No missing values — clean dataset ✓

Inner join guarantees that every row in the output has both a valid stage label *and* a complete set of pathway scores — a prerequisite for any supervised machine learning analysis.

Why does the stage column come first? Convention: the target variable (label) is placed as the first column, followed by features. This matches standard ML dataset formats (e.g. scikit-learn, XGBoost CSV input).

Big Picture — Plain English

The pathway scores (from Steps 1–7) and the stage labels (from Steps 8–9) were built from two separate files. This step joins them by matching patient IDs, keeping only patients who appear in both files. The result is the final analysis-ready dataset: each row has a stage label and a complete set of pathway scores with no missing values.

11. Step 11 — Robust Download with 3-Method Fallback Chain

Step 11: Three download methods in order

What it does: Attempts to deliver the output CSVs to the user using three progressively more reliable methods.

```
1 # Method 1: Standard Colab files.download()
2 # Method 2: Copy to Google Drive (if mounted)
3 # Method 3: Base64 inline HTML download link
4 for fname in files_to_download:
5     if not method1_colab(fname):
6         if not method2_drive(fname):
7             method3_base64(fname)
```

Why this choice?

Why three methods?

1. **Method 1 (files.download):** Works in most Colab sessions but can fail silently in restricted environments, headless runs, or when the browser popup is blocked.
2. **Method 2 (Google Drive):** Colab has persistent access to the user's Drive. Copying to MyDrive/ provides a permanent copy even if the runtime disconnects.
3. **Method 3 (Base64 HTML link):** Works in any Jupyter-compatible environment. Encodes the file as a data URI embedded directly in the notebook output — no server call required.

The fallback chain ensures the user always gets their files regardless of browser settings, network restrictions, or runtime configuration. The computation (Steps 1–10) is never re-run regardless of which download method succeeds.

Big Picture — Plain English

The two output CSV files are saved on the Colab server's temporary disk. This step delivers them to the user using three methods in order: direct browser download, copy to Google Drive, and an inline HTML download link. If one method fails, the next is tried automatically. This ensures the output files are always accessible regardless of the runtime environment.

Summary Table

Step	Operation	Key Parameter	Reason
1	Load expression	Median imputation	Robust to outliers
2	Load pathways	$10 \leq k \leq 1000$	Statistical stability; remove non-specific sets
3	Boolean masks	<code>dtype=bool</code>	8× memory savings
4	Rank samples	<code>method=average</code>	Exact Barbie 2009 spec
5	ssGSEA score	$\alpha = 0.25$	Standard ssGSEA weighting
6	Parallel batches	<code>batch=100</code>	Balance overhead vs. parallelism
7	Normalise $[0, 1]$	Per-pathway min-max	Comparable scales across pathways
8	Load clinical	<code>low_memory=False</code>	Consistent mixed-type dtypes
9	8-phase stage	OS threshold = 180d	Standard early-event clinical cutoff
10	Inner join	Common samples only	No missing values in final matrix
11	Robust download	3-method chain	Guarantees delivery in all environments

Final Output Structure

$\underbrace{\text{sample_id}}_{\text{index}}$	$\underbrace{\text{stage}}_{\in \{0, \dots, 7\}}$	$\underbrace{\text{Pathway}_1 \text{ Pathway}_2 \dots \text{Pathway}_P}_{\in [0, 1], P \approx 1,400 \text{ Reactome pathways}}$
---	---	--

Each row is one TCGA BRCA patient sample.

Each numeric cell is a normalised ssGSEA pathway activity score.

The **stage** column encodes 8 clinically defined phenotype groups.